AI - Assignment # 1
Abdullah Sonija
24k-0013

Question # 1

Self Driving Car :

a. The core objective is to achieve level 5 autonomy, where a vehicle navigates complex environments without human interaction. It reflects AI goals by mimicking human perception i.e. Computer vision & decision making

b. Sensing Layer : Collects raw data from cameras
↓
Perception layer: Uses neural networks to identify objects, lanes, and traffic lights.
↓
Planning layer : Predict trajectory of other agents & calculates the safest path.
↓
Actuation : Convert decisions into steering, braking, and acceleration commands.

c. Key Component includes the FSD chips (hardware), neural networks, and fleet learning. These interact by feeding high speed data into models that produce real-time driving behaviour.

d. Real time Impact: It has revolutionized the automotive Industry but represent narrow intelligence (ANI). It excels at driving but cannot perform tasks outside that domain, such as playing chess or conversing.

e. Limitation:

Technical (Occlusion / bad weather)
Ethical (trolley problem / liability),
Practical (regulatory hurdles). These limits full public acceptance until 99.99% reliability is proven.

f. Future Improvement: Integrating V2X (vehicle-to-everything) communication and explainable AI to make decisions transparent and socially responsible.

Question # 2

AI in transportation (Traffic Management)

a. Emerging Technology: AI powered intelligent traffic management system.

b. Mechanism: Uses computervision and IOT sensors to analyze traffic flow in real time.

c. Analysis:
Advantages: Reduces travel time
Challanges: High infrastructure cost & data privacy.
Ethical Issues: Surveillance concerns regarding tracking vehicle movement

d. Improvements: Integration with emergency services to automatically clear path for ambulance.

Question # 3

a. PEAS Description

Agent	Performance	Environment	Actuators	Sensors
Resume Scaner	Accuracy, Fairness	Applicant Database	Displaying shortlist	Parser, NLP Engine
Flood detector	Early Warning Accuracy	River/ Climate Data	Alert, sirens	water level/ Rain sensors

b. Environment Table

Agent	Environment types
(i) Resume Screener (RS)	Fully Observable, Deterministic, Episodic, Static Discrete, Single
(ii) Flood Detector (FD)	Partially Observable, Stochastic, Sequential, Dynamic, Continuous, Multi

(i) (RS) reasoning. The environment is static because the database of resumes does not change while agent is processing a single batch.

(ii) (FD) reasoning: The environment is stochastic because the weather patterns & water levels involve uncertainty and cannot be 100% predicted

Question # 4

- Implicit Agent: The automated retrival system (Robot).
- State Space: Co-ordinates (x, y) of the robot in the girid and the status of the book (not picked/picked).
- Intial State: Robot at fixed location (x_0, y_0), book at (x_b, y_b).
- Actions: Move {North, south, east, West}, Pick up book.
- Transition Model: Moving changes (x, y) coordinates; 'Pickup' changes book status if at (x_b, y_b)
- Goal Test: Robot at (x_b, y_b) And book is 'Picked up'.
- Path Cost: Each move has a cost of 1 unit of time/energy.
- Solution: A sequence of moves from start to shelf that avoids blocked aisles.

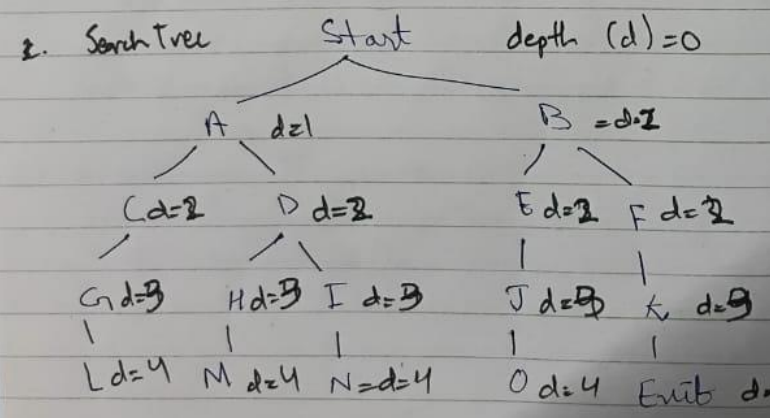
Question # 5

Uniformed Search (Drone in Cave)

1. Search Strategy: Iterative Deepening Search (IDS).
It explores like DFS but gradually increases the depth limit (L=0, 1, 1, 2, ...)

2. Appropriateness:
Memory: IDS only needs to store current path (O(bd)), which suits memory.
Unknown Depth: It is Complete - it will eventually find the exit regardless of depth
Dead Ends: IDS avoids getting trapped forever in infinite or very deep dead-end tunnels because of depth-limit.

3. Search Tree



Question 6:

```
maze = [
    ['S', 0, 0, 1, 0],
    [1, 0, 1, 0, 0],
    [0, 0, 0, 0, 'G'],
    [1, 1, 0, 1, 1]
]

def get_start_goal(maze):
    start, goal = None, None
    for r in range(len(maze)):
        for c in range(len(maze[0])):
            if maze[r][c] == 'S': start = (r, c)
            if maze[r][c] == 'G': goal = (r, c)
    return start, goal

def dls(current, goal, limit, path, visited):
    if current == goal: return path
    if limit <= 0: return None

    visited.add(current)
    r, c = current
    for dr, dc in [(-1,0), (1,0), (0,-1), (0,1)]:
        nr, nc = r + dr, c + dc
        if 0 <= nr < len(maze) and 0 <= nc < len(maze[0]) and \
                 maze[nr][nc] != 1 and (nr, nc) not in visited:
            result = dls((nr, nc), goal, limit - 1, path + [(nr, nc)],
visited.copy())
            if result: return result
    return None

def iterative_deepening(start, goal):
    for depth in range(200):
        result = dls(start, goal, depth, [start], set())
        if result: return result, depth
    return None, None

start, goal = get_start_goal(maze)
path, depth = iterative_deepening(start, goal)
print(f"IDS Path: {path} found at depth {depth}")
```

```
"""
Iterative Deepening Search (IDS) is more suitable than Depth-First Search (DFS)
here because:
    Optimality: IDS finds the shortest path (nearest goal) in an unweighted grid,
whereas DFS might take a very long, circuitous route.
    Completeness: If the maze were infinite or had cycles, DFS could get lost
forever, but IDS ensures every level is checked before moving deeper.
"""
```